Coding: CSharp (C#) Standards

CSharp

```
---
applyTo: "**/*.cs"
---

# C#.NET Best practices
C# best practices cover a wide range of topics, including functional programming techniques, coding style conventions, and exception handling best practices.
This guide provides an overview of these topics to help developers write clean, maintainable, and efficient C# code that AI should follow.

## Table of Contents
1. [Functional Programming Techniques](#functional-programming-techniques)
   - [Pattern Matching](#pattern-matching)
   - [Discards](#discards)
2. [Coding Style Conventions](#coding-style-conventions)
   - [Identifier Naming Rules and Conventions](#identifier-naming-rules-and-conventions)
   - [Common C# Code Conventions](#common-c-code-conventions)
   - [Exception Handling Best Practices](#exception-handling-best-practices)

## Functional Programming Techniques

### Pattern Matching

Pattern matching is a feature that allows for more expressive and concise code by testing expressions against patterns and taking action when a pattern matches.

#### Types of Pattern Matching

1. **Type Patterns**
   - Test if an expression is of a specific type
   - Can declare a variable of the matched type

   ```csharp
   if (obj is string s)
   {
       // Use the variable s as a string
       Console.WriteLine(s.Length);
   }
   ```

2. **Constant Patterns**
   - Match expressions against constant values

   ```csharp
   if (amount is 0)
   {
       return "No balance";
   }
   ```

3. **Relational Patterns**
   - Use comparison operators in patterns (>=, <=, >, <)

   ```csharp
   string GetWaterState(int temperature) => temperature switch
   {
       < 0 => "Solid",
       0 => "Solid/Liquid",
       > 0 and < 100 => "Liquid",
       100 => "Liquid/Gas",
       > 100 => "Gas"
   };
   ```

4. **Logical Patterns**
   - Combine patterns using logical operators (and, or, not)

   ```csharp
   if (input is > 0 and < 10)
   {
       Console.WriteLine("Single digit positive number");
   }
   ```

```

5. **Property Patterns**
   - Match on properties of an object

   ```csharp
   if (person is { Name: "John", Age: >= 18 })
   {
       Console.WriteLine("Adult named John");
   }
   ```

6. **Tuple Patterns**
   - Match on tuple elements

   ```csharp
   var point = (2, 3);
   string quadrant = point switch
   {
       (0, 0) => "Origin",
       (> 0, > 0) => "First quadrant",
       (< 0, > 0) => "Second quadrant",
       (< 0, < 0) => "Third quadrant",
       (> 0, < 0) => "Fourth quadrant",
       _ => "On an axis"
   };
   ```

7. **Positional Patterns**
   - Match on the structure returned by deconstruction

   ```csharp
   static string Classify(Point point) => point switch
   {
       (0, 0) => "Origin",
       (var x, 0) => $"X-axis at {x}",
       (0, var y) => $"Y-axis at {y}",
       _ => "Elsewhere"
   };
   ```

8. **List Patterns**
   - Match on sequence elements

   ```csharp
   int[] numbers = { 1, 2, 3 };
   string result = numbers switch
   {
       [] => "Empty array",
       [1, 2, 3] => "Array with 1, 2, 3",
       [1, ..] => "Array starting with 1",
       [.., 3] => "Array ending with 3",
       _ => "Other array"
   };
   ```

Pattern Matching Contexts

1. **is Expression**
   - Test if an expression matches a pattern

   ```csharp
   if (input is int i && i > 0)
   {
       // Use i as a positive integer
   }
   ```

2. **switch Expression**
   - Evaluate an expression against multiple patterns

   ```csharp
   string GetSize(int size) => size switch
   {
       <= 0 => "Invalid",
       < 10 => "Small",
       < 50 => "Medium",
       < 100 => "Large",
       _ => "Extra Large"
   };
   ```

3. **switch Statement**
   - Traditional switch with pattern matching

```csharp
switch (shape)
{
    case Circle c when c.Radius > 10:
        Console.WriteLine("Large circle");
        break;
    case Rectangle r when r.Width == r.Height:
        Console.WriteLine("Square");
        break;
    case Rectangle r:
        Console.WriteLine("Rectangle");
        break;
    default:
        Console.WriteLine("Unknown shape");
        break;
}
```

Best Practices for Pattern Matching

1. Use pattern matching to simplify complex conditional logic
2. Prefer switch expressions for transformations that map inputs to outputs
3. Use property patterns to avoid null checks and property access
4. Combine patterns for more expressive conditions
5. Use discard pattern (`_`) for catch-all cases
6. Order patterns from most specific to most general

Discards

Discards are temporary, write-only variables used in assignments when you want to ignore particular values. They are represented by the underscore character (`_`).

Use Cases for Discards

1. **Tuple and Object Deconstruction**
   - Ignore specific elements in deconstruction

   ```csharp
   var (name, _, _, country) = GetPersonDetails(); // Ignore age and address
   ```

2. **Pattern Matching**
   - Discard parts of matched patterns

   ```csharp
   if (person is { Name: "John", Age: _ })
   {
       // We only care about the name being "John"
   }
   ```

3. **Out Parameters**
   - Ignore out parameters you don't need

   ```csharp
   if (int.TryParse(input, out _)) // We only care if parsing succeeded
   {
       Console.WriteLine("Valid integer");
   }
   ```

4. **Standalone Discard**
   - Explicitly ignore a value

   ```csharp
   _ = Task.Run(async () =>
   {
       await LongRunningOperation();
       // We're not awaiting the task
   });
   ```

5. **Lambda Expressions**
   - Ignore unused parameters

   ```csharp
   Func<int, int, int> add = (x, _) => x + 5; // Second parameter is ignored
   ```

Best Practices for Discards

1. Use discards to indicate intentionally ignored values
2. Use multiple discards in the same scope (they're not actual variables)

3. Use discards to improve code readability by removing unused variables
4. Consider using named variables instead of discards if the value might be needed later
5. Use discards in LINQ queries to ignore elements in projections

Coding Style Conventions

Identifier Naming Rules and Conventions

C# has specific naming conventions that help create consistent, readable code.

Naming Rules

1. **Identifiers must:**
   - Begin with a letter or underscore
   - Contain only letters, digits, or underscores
   - Not be a C# keyword (unless using `@` prefix)

2. **Case Sensitivity**
   - C# is case-sensitive (`count` and `Count` are different identifiers)

Naming Conventions by Identifier Type

1. **Namespace Names**
   - PascalCase
   - Use company/organization name as root namespace
   - Use plural form for namespaces containing types (e.g., `System.Collections`)

   ```csharp
   namespace Company.Product.Module
   ```

2. **Class and Type Names**
   - PascalCase
   - Use nouns or noun phrases
   - Avoid prefixes (e.g., "C" or "I" except for interfaces)

   ```csharp
   public class CustomerService
   ```

3. **Interface Names**
   - PascalCase with "I" prefix
   - Use adjectives, nouns, or occasionally verbs

   ```csharp
   public interface IComparable
   ```

4. **Method Names**
   - PascalCase
   - Use verbs or verb phrases

   ```csharp
   public void CalculateTotal()
   ```

5. **Property Names**
   - PascalCase
   - Use nouns or adjectives

   ```csharp
   public string FirstName { get; set; }
   ```

6. **Event Names**
   - PascalCase
   - Use verb phrases
   - Use present or past tense based on before/after occurrence

   ```csharp
   public event EventHandler DataLoaded;
   ```

7. **Field Names**
   - Private/internal fields: camelCase with underscore prefix
   - Public fields (rare): PascalCase

   ```csharp
   private string _firstName;
   public const int MaxLength = 100;
   ```

8. **Parameter Names**

```
   - camelCase

   ```csharp
   public void ProcessOrder(Order order, bool expedite)
   ```

9. **Local Variable Names**
   - camelCase

   ```csharp
   var customerCount = customers.Count;
   ```

10. **Type Parameter Names**
    - PascalCase with "T" prefix

    ```csharp
    public class List<TItem>
    ```

11. **Constant Names**
    - PascalCase

    ```csharp
    public const string DefaultName = "Unknown";
    ```

12. **Enum Names and Values**
    - Enum types: PascalCase (singular)
    - Enum values: PascalCase

    ```csharp
    public enum Color
    {
        Red,
        Green,
        Blue
    }
    ```

Abbreviations and Acronyms

1. Treat acronyms as words for casing purposes:
   - `HttpRequest` (not `HTTPRequest`)
   - `XmlDocument` (not `XMLDocument`)
   - `GetId` (not `GetID`)

2. Two-letter acronyms use all uppercase:
   - `IOStream`
   - `UIControl`

Best Practices for Naming

1. Choose descriptive names that convey intent
2. Favor readability over brevity
3. Avoid Hungarian notation (e.g., `strName`)
4. Don't include the type in the name (e.g., `nameString`)
5. Use meaningful names for generic type parameters (e.g., `TKey` instead of just `T`)
6. Use consistent naming across related identifiers
7. Avoid using language keywords as identifiers

Common C# Code Conventions

Following consistent code conventions improves readability and maintainability.

Layout Conventions

1. **Indentation**
   - Use 4 spaces for indentation (not tabs)
   - Indent all control structures

2. **Line Breaks**
   - One statement per line
   - Break long statements into multiple lines for readability
   - Align wrapped parameters with the first parameter

   ```csharp
   var result = longExpression1
                + longExpression2
                + longExpression3;
   ```

3. **Braces**
```

- Place opening braces on a new line
- Align closing braces with the opening statement

```csharp
if (condition)
{
    DoSomething();
}
```

4. **Spacing**
   - Use a space after keywords like `if`, `for`, `while`
   - Use spaces around operators
   - No space between method name and parenthesis
   - No space inside parentheses

   ```csharp
   if (x == y)
   {
       DoSomething(x, y);
   }
   ```

5. **Member Organization**
   - Group members by type (fields, properties, constructors, methods)
   - Order members by accessibility (public, protected, internal, private)
   - Implement interface members together

Commenting Conventions

1. **Comment Style**
   - Use `//` for single-line comments
   - Use `/* */` for multi-line comments
   - Use XML documentation comments (`///`) for public APIs

2. **Documentation Comments**
   - Document all public types and members
   - Include `<summary>`, `<param>`, `<returns>`, `<exception>` tags as appropriate

   ```csharp
   /// <summary>
   /// Calculates the total price including tax.
   /// </summary>
   /// <param name="price">The base price.</param>
   /// <param name="taxRate">The tax rate as a percentage.</param>
   /// <returns>The total price including tax.</returns>
   public decimal CalculateTotalPrice(decimal price, decimal taxRate)
   ```

3. **Comment Content**
   - Focus on why, not what (the code shows what)
   - Explain complex algorithms or business rules
   - Document assumptions and constraints
   - Keep comments up-to-date with code changes

Language Guidelines

1. **String Usage**
   - Use string interpolation for string composition

   ```csharp
   var message = $"Hello, {name}! Today is {date:d}.";
   ```

   - Use verbatim strings for multi-line or path strings

   ```csharp
   var path = @"C:\Program Files\App";
   ```

2. **var Usage**
   - Use `var` when the type is obvious from the right side
   - Avoid `var` when the type is not clear from the initialization

   ```csharp
   var customers = new List<Customer>(); // Good
   var result = GetResult(); // Potentially unclear
   ```

3. **Null Checking**
   - Use null-conditional operators (`?.`, `?[]`) for safe member access
   - Use null-coalescing operator (`??`) for providing default values
   - Use null-coalescing assignment (`??=`) for assigning default values

```csharp
var name = customer?.Name ?? "Unknown";
orders ??= new List<Order>();
```

4. **Exception Handling**
   - Use specific exception types rather than `Exception`
   - Only catch exceptions you can handle
   - Use `finally` blocks for cleanup code

```csharp
try
{
    ProcessFile(path);
}
catch (FileNotFoundException ex)
{
    LogError($"File not found: {ex.Message}");
}
finally
{
    CleanupResources();
}
```

5. **LINQ Usage**
   - Prefer method syntax for simple queries
   - Use query syntax for complex queries with multiple operations
   - Break long LINQ queries into multiple lines

```csharp
// Method syntax
var adults = people.Where(p => p.Age >= 18).ToList();

// Query syntax
var result = from p in people
             where p.Age >= 18
             orderby p.LastName, p.FirstName
             select new { p.FirstName, p.LastName };
```

6. **Async/Await**
   - Use async/await instead of Task continuations
   - Name async methods with "Async" suffix
   - Avoid async void except for event handlers
   - Use `Task.ConfigureAwait(false)` in library code

```csharp
public async Task<User> GetUserAsync(int userId)
{
    var data = await _repository.GetUserDataAsync(userId)
                                .ConfigureAwait(false);
    return new User(data);
}
```

Exception Handling Best Practices

Proper exception handling is crucial for robust applications.

Things to Avoid When Throwing Exceptions

1. **Don't Use Exceptions for Normal Flow Control**
   - Exceptions should represent exceptional conditions
   - Use return values or out parameters for expected failure cases

```csharp
// Bad
try
{
    var user = GetUser(id);
}
catch (UserNotFoundException)
{
    // Handle missing user
}

// Good
if (TryGetUser(id, out var user))
{
    // Use user
}
```

```
   else
   {
       // Handle missing user
   }
   ```

2. **Don't Throw Generic Exceptions**
   - Avoid throwing `Exception`, `SystemException`, `ApplicationException`
   - Use or create specific exception types

   ```csharp
   // Bad
   throw new Exception("User not found");

   // Good
   throw new UserNotFoundException($"User with ID {userId} not found");
   ```

3. **Don't Catch Everything Without Rethrowing**
   - Avoid empty catch blocks
   - Don't swallow exceptions without handling them

   ```csharp
   // Bad
   try
   {
       ProcessData();
   }
   catch (Exception)
   {
       // Silently failing
   }

   // Good
   try
   {
       ProcessData();
   }
   catch (Exception ex)
   {
       LogError(ex);
       throw; // Rethrow if you can't handle it
   }
   ```

4. **Don't Throw From Finally Blocks**
   - Exceptions from finally blocks override exceptions from try/catch
   - Use safe cleanup code in finally blocks

   ```csharp
   // Bad
   try
   {
       ProcessFile();
   }
   finally
   {
       if (file != null)
       {
           throw new Exception("Error closing file"); // Bad
       }
   }

   // Good
   try
   {
       ProcessFile();
   }
   finally
   {
       file?.Close(); // Safe cleanup
   }
   ```

5. **Don't Include Sensitive Information in Exception Messages**
   - Exception messages may be exposed to users
   - Avoid including passwords, connection strings, etc.

   ```csharp
   // Bad
   throw new ConnectionException($"Failed to connect to {server} with password {password}");

   // Good
```

```
   throw new ConnectionException($"Failed to connect to {server}");
   ```

Exceptions in Task-Returning Methods

1. **Unhandled Exceptions in Async Methods**
   - Exceptions in async methods are captured and placed on the returned Task
   - The exception is thrown when the Task is awaited

   ```csharp
   public async Task ProcessDataAsync()
   {
       await Task.Delay(100);
       throw new InvalidOperationException(); // Captured in returned Task
   }

   // Later:
   try
   {
       await ProcessDataAsync(); // Exception is thrown here
   }
   catch (InvalidOperationException ex)
   {
       // Handle exception
   }
   ```

2. **Multiple Exceptions in Tasks**
   - When awaiting `Task.WhenAll`, only the first exception is thrown
   - Access the `Exception.InnerExceptions` property to see all exceptions

   ```csharp
   try
   {
       await Task.WhenAll(task1, task2, task3);
   }
   catch (Exception ex) when (ex is AggregateException aex)
   {
       foreach (var inner in aex.InnerExceptions)
       {
           LogError(inner);
       }
   }
   ```

3. **Handling Exceptions in Fire-and-Forget Tasks**
   - Use a continuation to handle exceptions in fire-and-forget tasks

   ```csharp
   Task.Run(async () =>
   {
       await LongRunningOperation();
   }).ContinueWith(t =>
   {
       if (t.IsFaulted)
       {
           LogError(t.Exception);
       }
   });
   ```

4. **Cancellation and Exceptions**
   - Check for cancellation before throwing exceptions
   - Use `OperationCanceledException` for cancellation

   ```csharp
   public async Task ProcessAsync(CancellationToken cancellationToken)
   {
       cancellationToken.ThrowIfCancellationRequested();

       // Start work
       await Task.Delay(1000, cancellationToken);

       // Check again before continuing
       cancellationToken.ThrowIfCancellationRequested();

       // Continue work
   }
   ```

5. **Exception Filters for Logging**
   - Use exception filters for logging without catching
```
```

```csharp
try
{
    await ProcessDataAsync();
}
catch (Exception ex) when (LogException(ex))
{
    // This block never executes
}

private bool LogException(Exception ex)
{
    Logger.LogError(ex);
    return false; // Continue to the next catch clause
}
```

Best Practices for Exception Handling

1. **Be Specific About What You Catch**
   - Catch specific exception types, not base classes
   - Only catch exceptions you can handle

2. **Include Context in Exception Messages**
   - Provide enough information to understand what went wrong
   - Include relevant parameter values (except sensitive data)

3. **Preserve the Original Exception**
   - Use `throw;` instead of `throw ex;` to preserve stack trace
   - Use inner exceptions when wrapping exceptions

   ```csharp
   try
   {
       ProcessFile(path);
   }
   catch (IOException ex)
   {
       throw new ApplicationException("Error processing file", ex);
   }
   ```

4. **Clean Up Resources**
   - Use `using` statements for disposable resources
   - Use `finally` blocks for cleanup when `using` isn't applicable

   ```csharp
   using (var connection = new SqlConnection(connectionString))
   {
       // Use connection
   } // Automatically disposed
   ```

5. **Design for Exceptions**
   - Make your API fault-tolerant
   - Document exceptions in XML comments
   - Consider providing both exception-throwing and TryX methods

   ```csharp
   /// <summary>
   /// Gets a user by ID.
   /// </summary>
   /// <param name="userId">The user ID.</param>
   /// <returns>The user.</returns>
   /// <exception cref="UserNotFoundException">Thrown when the user doesn't exist.</exception>
   public User GetUser(int userId)
   {
       // Implementation
   }

   /// <summary>
   /// Tries to get a user by ID.
   /// </summary>
   /// <param name="userId">The user ID.</param>
   /// <param name="user">The user, if found.</param>
   /// <returns>True if the user was found; otherwise, false.</returns>
   public bool TryGetUser(int userId, out User user)
   {
       // Implementation
   }
   ```

6. **Global Exception Handling**

- Implement global exception handlers for unhandled exceptions
- Log all unhandled exceptions
- Provide user-friendly error messages

```csharp
// ASP.NET Core example
app.UseExceptionHandler(errorApp =>
{
    errorApp.Run(async context =>
    {
        var exceptionHandlerPathFeature = context.Features.Get<IExceptionHandlerPathFeature>();
        var exception = exceptionHandlerPathFeature.Error;

        Logger.LogError(exception, "Unhandled exception");

        context.Response.StatusCode = 500;
        await context.Response.WriteAsync("An unexpected error occurred.");
    });
});
```